DEV_IMPL_GUIDE - EM-01 Franchise Management

Use with `DD_EM-01-Franchise_Management-v1.0.md`.

This guide explains how EM-01 executes at runtime so a mid-level developer can implement the module without guessing ownership, validations, or integration flow.

1. Runtime Components

Component	Responsibility
EM API pod	Receives franchise commands and read requests. Can be the same deployable as EM-02 at first.
EM service layer	Applies franchise lifecycle, assignment, and resolution rules.
EM DB schema	Stores franchise master, boundaries, assignments, contacts, external refs, lifecycle events, and outbox rows.
RLM client	Validates HomePass ids and reads HomePass detail.
EM-02 client	Validates contractor/staff/team references when provided.
Auth/RBAC client	Validates user identity and later resolves franchise-scoped permissions.
Kafka outbox dispatcher	Publishes franchise events after DB commit.
Reporting consumer	Consumes events to build franchise dimensions.

For a first medium deployment, EM-01 does not need its own pod. It can run inside the existing EM/business-services application, provided routes, schema, metrics, and ownership remain clear.

2. Before Implementation Starts

Create these tables:

```
em_franchise
em_franchise_boundary
em_franchise_homepass_assignment
em_franchise_external_ref
em_franchise_contact
em_franchise_lifecycle_event
outbox_event
idempotency_key
```

Seed or configure these values:

```
franchise_status: DRAFT, ACTIVE, SUSPENDED, RETIRED
franchise_type: INTERNAL, EXTERNAL_PARTNER, DIRECT_BRANCH, DEALER
commercial_segment: B2C, B2B, MIXED
assignment_role: PRIMARY, SECONDARY, B2B_ONLY, B2C_ONLY, TEMPORARY
```

Minimum indexes:

- unique `(operator_code, franchise_code)` on `em_franchise`;
- `(operator_code, status)` on `em_franchise`;
- `(homepass_id, active)` on `em_franchise_homepass_assignment`;
- partial unique primary assignment index for active `PRIMARY` assignment per `(operator_code, homepass_id, commercial_segment)` if `commercial_segment` is materialized;
- `(franchise_id, active)` on assignments and contacts;
- `(system_code, external_code)` on external refs.

3. Flow 1 - Create Franchise

3.1 Caller Sends Request

Backoffice sends:

```
POST /api/franchises
Authorization: Bearer <jwt>
Idempotency-Key: create-franchise-wik-nrb-002
Content-Type: application/json

{
  "operatorCode": "WIK",
  "franchiseCode": "WIK-NRB-002",
  "displayName": "Nairobi 002 - Lavington/Kileleshwa",
  "shortName": "NRB 002",
  "countryCode": "KE",
  "primaryCity": "Nairobi",
  "franchiseType": "EXTERNAL_PARTNER",
  "commercialSegment": "B2C",
  "contractorId": "contractor_ke_nairobi_a",
  "managerUserId": "kc_user_franchise_mgr_002",
  "defaultLanguage": "en-KE",
  "supportedLanguages": ["en-KE", "sw-KE"],
  "defaultCurrency": "KES",
  "effectiveFrom": "2026-06-01"
}
```

3.2 Request Lands On EM API

Validate:

Field	Validation
`operatorCode`	Required and must match caller scope.
`franchiseCode`	Required, uppercase business code, unique per operator.
`displayName`	Required.
`countryCode`	Required.
`franchiseType`	Must exist in configured enum/catalog.
`commercialSegment`	Must exist in configured enum/catalog.
`contractorId`	Optional, but if provided must resolve through EM-02.
`managerUserId`	Optional, but if provided must resolve through Auth/EM-02.
`effectiveFrom`	Required.
`Idempotency-Key`	Required for create.

3.3 EM Service Executes

1. Check idempotency key.
2. Hash the request body.
3. If the same key and same hash already succeeded, return the stored response.
4. If the same key exists with a different hash, return `409 CONFLICT`.
5. Check `(operator_code, franchise_code)` is not already used.
6. If `contractorId` exists, call EM-02:

```
GET /api/contractors/contractor_ke_nairobi_a
Authorization: Bearer <service-token>
```

7. If `managerUserId` exists, validate the user is known and active.
8. Start one DB transaction.
9. Insert `em_franchise` with `status = DRAFT`.
10. Insert `em_franchise_lifecycle_event` with `event_type = FRANCHISE_CREATED`.
11. Insert outbox event `FranchiseCreated`.
12. Store idempotency response.
13. Commit.

3.4 EM API Returns

```
201 Created

{
```

```
  "franchiseId": "fr_01J_NRB_002",
  "franchiseCode": "WIK-NRB-002",
  "displayName": "Nairobi 002 - Lavington/Kileleshwa",
  "status": "DRAFT"
}
```

4. Flow 2 - Activate Franchise

4.1 Caller Sends Request

```
POST /api/franchises/fr_01J_NRB_002/activate
Authorization: Bearer <jwt>
Idempotency-Key: activate-franchise-wik-nrb-002
Content-Type: application/json

{
  "reasonCode": "INITIAL_SETUP",
  "comment": "Franchise validated by commercial operations."
}
```

4.2 EM Service Executes

1. Check idempotency key and request hash.
2. Lock franchise row:

```
SELECT *
FROM em_franchise
WHERE franchise_id = :franchiseId
FOR UPDATE;
```

3. Validate current status is `DRAFT` or `SUSPENDED`.
4. Validate `effective_from <= today`.
5. If the franchise is `EXTERNAL_PARTNER` and operator config requires contractor linkage, verify `contractor_id` is present and active in EM-02.
6. Update `status = ACTIVE`.
7. Insert lifecycle event `FRANCHISE_ACTIVATED`.
8. Insert outbox event `FranchiseActivated`.
9. Store idempotency response.
10. Commit.

4.3 Downstream Effect

The outbox dispatcher publishes:

```
{
  "eventType": "FranchiseActivated",
  "operatorCode": "WIK",
  "aggregateId": "fr_01J_NRB_002",
  "payload": {
    "franchiseCode": "WIK-NRB-002",
    "displayName": "Nairobi 002 - Lavington/Kileleshwa",
    "status": "ACTIVE"
  }
}
```

Consumers:

- SALES refreshes active franchise lookup.
- RLM refreshes franchise display/projection for HomePass responses.
- REP updates franchise dimension.
- Frontend caches invalidate active franchise list.

5. Flow 3 - Assign HomePass To Franchise

5.1 Caller Sends Request

```
POST /api/franchises/fr_01J_NRB_002/homepass-assignments
Authorization: Bearer <jwt>
Idempotency-Key: hp-franchise-hp01-nrb002
Content-Type: application/json

{
  "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "assignmentRole": "PRIMARY",
  "assignmentReason": "MIGRATED",
  "effectiveFrom": "2026-06-01"
}
```

5.2 EM Service Validates

1. Check idempotency key and request hash.
2. Load franchise and verify `status = ACTIVE`.
3. Call RLM to validate HomePass exists:

```
GET /api/homepasses/hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P
Authorization: Bearer <service-token>
```

4. Reject if RLM returns `404`.
5. Validate `assignmentRole`.
6. If role is `PRIMARY`, check there is no other active primary assignment for the same HomePass and commercial segment.
7. Start one DB transaction.
8. Insert `em_franchise_homepass_assignment`.
9. Insert lifecycle event `FRANCHISE_HOMEPASS_ASSIGNED`.
10. Insert outbox event `FranchiseHomePassAssigned`.
11. Store idempotency response.
12. Commit.

5.3 Response

```
201 Created

{
  "assignmentId": "fha_01J_HP_9988",
  "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "franchiseId": "fr_01J_NRB_002",
  "assignmentRole": "PRIMARY",
  "active": true
}
```

5.4 RLM Projection

RLM should not manually edit franchise data. It can consume the event or call EM-01 when returning HomePass detail:

```
{
  "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "statusCode": "RFS",
  "franchiseAssignments": [
    {
      "franchiseId": "fr_01J_NRB_002",
      "franchiseCode": "WIK-NRB-002",
      "assignmentRole": "PRIMARY"
    }
  ]
}
```

6. Flow 4 - Resolve Franchise During Lead Capture

6.1 Caller Sends Request

SALES-01 or FE-APP-02 backend client sends:

```
POST /api/franchises/resolve
```

```
Authorization: Bearer <service-token>
Idempotency-Key: resolve-franchise-lead-01J
Content-Type: application/json

{
  "operatorCode": "WIK",
  "contextType": "LEAD_CAPTURE",
  "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "geo": {
    "lat": -1.2831,
    "lng": 36.782
  },
  "commercialSegment": "B2C",
  "requestingUserId": "kc_user_sales_04"
}
```

6.2 EM Service Resolves

Resolution order:

1. If `homepassId` is present, find active HomePass assignment.
2. If one active `PRIMARY` assignment exists, return it.
3. If multiple assignments exist, filter by `commercialSegment`.
4. If still multiple remain, return `AMBIGUOUS` with candidates.
5. If no HomePass assignment exists and PostGIS/boundary lookup is enabled, match `geo` against active boundaries.
6. If no match, return `UNRESOLVED`.

Response for resolved case:

```
{
  "resolutionStatus": "RESOLVED",
  "primaryFranchise": {
    "franchiseId": "fr_01J_NRB_002",
    "franchiseCode": "WIK-NRB-002",
    "displayName": "Nairobi 002 - Lavington/Kileleshwa"
  },
  "secondaryFranchises": [],
  "source": "HOMEPASS_ASSIGNMENT",
  "warnings": []
}
```

Response for ambiguous case:

```
{
  "resolutionStatus": "AMBIGUOUS",
  "primaryFranchise": null,
  "secondaryFranchises": [
    {
      "franchiseId": "fr_01J_NRB_002",
      "franchiseCode": "WIK-NRB-002"
    },
    {
      "franchiseId": "fr_01J_B2B_NRB",
      "franchiseCode": "WIK-B2B-NRB"
    }
  ],
  "source": "HOMEPASS_ASSIGNMENT",
  "warnings": ["Multiple active franchise assignments matched the context."]
}
```

Caller rule:

- SALES-01 may proceed automatically only for `RESOLVED`.
- For `AMBIGUOUS`, the sales supervisor or agent must select a valid franchise if policy allows.
- For `UNRESOLVED`, lead can be saved as draft/follow-up, but conversion to order should be blocked unless an override process exists.

7. Flow 5 - Suspend Or Retire Franchise

7.1 Suspend

Use suspend when the franchise should temporarily stop receiving new activity but historical data remains valid.

```
POST /api/franchises/fr_01J_NRB_002/suspend
Authorization: Bearer <jwt>
Idempotency-Key: suspend-franchise-wik-nrb-002
Content-Type: application/json

{
  "reasonCode": "COMMERCIAL_HOLD",
  "comment": "Commercial operations requested temporary hold."
}
```

Execution:

1. Lock franchise row.
2. Validate current status is `ACTIVE`.
3. Set status to `SUSPENDED`.
4. Insert lifecycle event.
5. Insert outbox event `FranchiseSuspended`.
6. Commit.

Downstream rules:

- SALES blocks new lead assignment/conversion to suspended franchise.
- FUL blocks new order attribution to suspended franchise.
- Existing customers/subscriptions remain unchanged.
- REP keeps historical facts grouped under the suspended franchise.

7.2 Retire

Use retire when the franchise should never receive new activity again.

Execution:

1. Lock franchise row.
2. Check active dependencies:
   - active HomePass assignments in EM-01;
   - open leads in SALES-01;
   - pending orders in FUL-02;
   - open work orders if WO has franchise attribution;
   - active contractor link in EM-02 if policy requires clean closure.
3. If active dependencies exist and no approved migration override is present, return `409 CONFLICT`.
4. If allowed, set `status = RETIRED` and `effective_to`.
5. Insert lifecycle event.
6. Insert outbox event `FranchiseRetired`.
7. Commit.

For v1, retire may be a synchronous API. When `EM-CFG-04 Approval Workflow Catalog` exists, retirement can require an approval workflow.

8. Bulk Import Flow

Use this for BH migration or initial setup.

Endpoint:

```
POST /api/franchise-homepass-assignments/bulk-import
Authorization: Bearer <jwt>
Idempotency-Key: bulk-franchise-assignment-nrb-002-batch-0001
Content-Type: application/json

{
```

```
  "operatorCode": "WIK",
  "sourceSystem": "BH",
  "batchRef": "NRB-002-2026-06-01-0001",
  "rows": [
    {
      "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
      "franchiseCode": "WIK-NRB-002",
      "assignmentRole": "PRIMARY",
      "effectiveFrom": "2026-06-01"
    }
  ]
}
```

Execution:

1. Validate idempotency key.
2. Validate batch size. Recommended max: 1000 rows.
3. Resolve all franchise codes.
4. Validate all HomePass ids by batched RLM API if available; otherwise call in chunks.
5. Validate no row violates primary assignment uniqueness.
6. Insert valid rows in one transaction for the batch.
7. Insert one outbox event per assignment or one compact batch event plus queryable detail rows. Prefer one event per assignment if downstream projections must be exact without extra calls.
8. Return success count and failed row details.

Response:

```
{
  "batchRef": "NRB-002-2026-06-01-0001",
  "status": "ACCEPTED",
  "inserted": 1,
  "failed": 0,
  "failures": []
}
```

If a row fails validation, choose one policy and document it:

- strict batch: reject entire batch;
- partial batch: insert valid rows and return failed rows.

For migration, strict batch is cleaner for reconciliation. For admin UI uploads, partial batch is more user-friendly.

9. SQL Migration Sketch

Use the project migration tool, but the schema should look like this conceptually:

```
CREATE TABLE em_franchise (
  franchise_id text PRIMARY KEY,
  operator_code text NOT NULL,
  franchise_code text NOT NULL,
  display_name text NOT NULL,
  short_name text,
  country_code text NOT NULL,
  primary_city text,
  franchise_type text NOT NULL,
  commercial_segment text NOT NULL,
  status text NOT NULL,
  contractor_id text,
  manager_user_id text,
  default_language text NOT NULL,
  supported_languages_json jsonb NOT NULL DEFAULT '[]'::jsonb,
  default_currency text NOT NULL,
  effective_from date NOT NULL,
  effective_to date,
  created_by_user_id text NOT NULL,
  created_at timestamptz NOT NULL,
  updated_at timestamptz NOT NULL,
  metadata_json jsonb NOT NULL DEFAULT '{}'::jsonb,
  UNIQUE (operator_code, franchise_code),
  CHECK (effective_to IS NULL OR effective_to > effective_from)
);
```

```
CREATE TABLE em_franchise_homepass_assignment (
  assignment_id text PRIMARY KEY,
  operator_code text NOT NULL,
  homepass_id text NOT NULL,
  franchise_id text NOT NULL REFERENCES em_franchise(franchise_id),
  assignment_role text NOT NULL,
  assignment_reason text NOT NULL,
  effective_from date NOT NULL,
  effective_to date,
  active boolean NOT NULL DEFAULT true,
  created_by_user_id text NOT NULL,
  created_at timestamptz NOT NULL,
  CHECK (effective_to IS NULL OR effective_to > effective_from)
);

CREATE INDEX idx_em_franchise_status
  ON em_franchise(operator_code, status);

CREATE INDEX idx_em_franchise_assignment_homepass
  ON em_franchise_homepass_assignment(operator_code, homepass_id, active);
```

If you need strict one-primary assignment, add a partial unique index. Exact expression depends on whether commercial segment is stored on the assignment table:

```
CREATE UNIQUE INDEX uq_homepass_primary_franchise
  ON em_franchise_homepass_assignment(operator_code, homepass_id)
  WHERE active = true AND assignment_role = 'PRIMARY';
```

10. Event Publishing

Events are not published inline during the HTTP request. Insert into outbox in the same transaction as the business change.

Outbox row example:

```
{
  "eventType": "FranchiseHomePassAssigned",
  "topic": "sophix.em.franchise-homepass-assigned",
  "aggregateType": "FranchiseHomePassAssignment",
  "aggregateId": "fha_01J_HP_9988",
  "operatorCode": "WIK",
  "payload": {
    "assignmentId": "fha_01J_HP_9988",
    "homepassId": "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "franchiseId": "fr_01J_NRB_002",
    "franchiseCode": "WIK-NRB-002",
    "assignmentRole": "PRIMARY",
    "active": true
  }
}
```

Consumers must be idempotent. If the same event is delivered twice, RLM/REP/SALES projections should keep one final state.

11. Error Handling

Case	Response
Duplicate franchise code	`409 CONFLICT` with code `FRANCHISE_CODE_EXISTS`.
Same idempotency key, different request hash	`409 CONFLICT` with code `IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST`.
Unknown contractor id	`422 UNPROCESSABLE_ENTITY` with code `CONTRACTOR_NOT_FOUND`.
Unknown HomePass id	`422 UNPROCESSABLE_ENTITY` with code `HOMEPASS_NOT_FOUND`.
Assign to inactive franchise	`409 CONFLICT` with code `FRANCHISE_NOT_ACTIVE`.
Duplicate active primary HomePass assignment	`409 CONFLICT` with code `PRIMARY_FRANCHISE_ALREADY_ASSIGNED`.
Retire franchise with active dependencies	`409 CONFLICT` with code `FRANCHISE_HAS_ACTIVE_DEPENDENCIES`.
Franchise resolution ambiguous	`200 OK` with `resolutionStatus = AMBIGUOUS`; not a server error.
Franchise resolution missing	`200 OK` with `resolutionStatus = UNRESOLVED`; not a server error.

12. Integration Rules

- SALES-01 should store franchise attribution from EM-01 at lead conversion time.
- FUL-02 should receive franchise attribution in the order payload and preserve it on order context.
- SUB-LM/BIL should not call EM-01 every time they display historical data. Store snapshots where needed for reporting stability.
- RLM should expose franchise assignment with HomePass detail, but EM-01 remains the owner.
- FE-APP-02 should not resolve franchises locally from GPS. It should call SALES-01 or EM-01 APIs.
- Reporting should build a franchise dimension from EM-01 events/read APIs.

13. Tests

Implement at least these tests:

Test	Expected result
Create franchise with valid payload	`201`, row created in `DRAFT`, outbox has `FranchiseCreated`.
Repeat same create with same idempotency key/body	Same response, no duplicate row.
Repeat same idempotency key with different body	`409`.
Duplicate franchise code for same operator	`409`.
Same franchise code for different operator	Allowed if multi-tenant deployment permits.
Activate draft franchise	Status becomes `ACTIVE`, lifecycle event and outbox event created.
Assign HomePass to active franchise	Assignment row created, event emitted.
Assign unknown HomePass	`422`, no assignment row.
Assign second primary franchise for same HomePass	`409`, unless overlap policy explicitly allows it.
Resolve by HomePass assignment	Returns `RESOLVED` and primary franchise.
Resolve ambiguous overlap	Returns `AMBIGUOUS`, not `500`.
Suspend franchise	New activity blocked; historical reads still work.
Retire with active assignment	`409` unless migration override exists.
Outbox dispatcher retries publish failure	Event remains pending and later publishes.

14. Implementation Checklist

1. Add migrations and table comments.
2. Add repository/service classes.
3. Add idempotency handling for command endpoints.
4. Add EM-02, RLM, and Auth clients.
5. Implement franchise create/read/update APIs.
6. Implement lifecycle APIs.
7. Implement HomePass assignment APIs.
8. Implement resolution API.
9. Implement outbox event insertion and publishing.
10. Add cache invalidation for active franchise lists.
11. Add integration tests with mocked RLM/EM-02 clients.
12. Add API docs/OpenAPI definitions.
13. Patch SALES-01, RLM-CFG-01, FE-APP-01, and FE-APP-02 references after DD approval.

15. What Not To Build In V1

- Do not build a full GIS engine unless boundary auto-resolution is required.
- Do not duplicate contractor workforce/capacity from EM-02.
- Do not duplicate sales territories from SALES-01.
- Do not put pricing logic in EM-01.
- Do not create Camunda BPMN for basic create/activate/suspend unless approval workflow is mandatory from day one.
- Do not make frontend apps cache franchise ownership permanently. Use API/cache with invalidation.